Lecture 22 : Project Estimation Techniques (Contd.)

Introduction

Accurate project estimation is a cornerstone of successful software development. While the basic COCOMO (Constructive Cost Model) provides a foundational approach by linking effort and development time primarily to product size, real-world projects are far more complex. Modern software development is influenced by a myriad of factors, from the reliability demands of the system to the tools and expertise of the development team. This lecture delves into the Intermediate and Complete COCOMO models, which enhance estimation accuracy by incorporating these critical parameters, moving beyond a simplistic size-only view to provide a more nuanced and realistic projection of project effort, cost, and duration. Understanding these advanced models is crucial for project managers to navigate the complexities of contemporary software engineering.

Core Concepts

Term	Definition	Significance
`Intermediate COCOMO`	An enhanced version of the Basic COCOMO model that refines initial effort estimates by considering 15 specific cost drivers or multipliers.	Provides more accurate estimations by accounting for various project-specific attributes beyond just product size, such as personnel capabilities and reliability requirements.
`Cost Drivers (Effort Multipliers)`	A set of 15 attributes categorized into personnel, product, computer, and project factors, each with a rating scale that influences project effort.	These factors allow for the adjustment of initial effort estimates, scaling them up or down based on the specific characteristics and challenges of a project.
	The product of all applicable cost driver values for a given project,	Crucial for tailoring the generic COCOMO estimate to the unique context of a project,

Term	Definition	Significance
`Effort Adjustment Factor (EAF)`	used to modify the initial effort estimate from the Basic COCOMO model.	reflecting how various attributes collectively impact the required effort.
`Complete COCOMO (Detailed COCOMO)`	The most advanced COCOMO model, designed to estimate the cost of each sub-system of a large project separately before summing them for a total cost.	Addresses the limitation of Basic and Intermediate COCOMO by recognizing that large systems are heterogeneous, allowing for more precise estimation of complex, multi-component projects.
`Development Modes`	Categories (Organic, Semi-Detached, Embedded) that classify projects based on size, team experience, and complexity, influencing COCOMO constants.	Determines the base constants (`c` and `k` for effort, and `exponent` for time) used in COCOMO equations, reflecting inherent differences in project types.
`KDSI (Kilo Delivered Source Instructions)`	A measure of product size, representing thousands of lines of delivered source code, used as a primary input for COCOMO calculations.	Serves as the fundamental input for estimating effort and time, with the assumption that larger codebases generally require more resources.

Detailed Analysis

1. Intermediate COCOMO: Incorporating Cost Drivers

The Basic COCOMO model, while foundational, simplifies project estimation by assuming that effort and development time depend solely on product size. However, modern software projects are influenced by numerous other parameters. Intermediate COCOMO addresses this limitation by introducing a set of 15 cost drivers (also known as effort multipliers) that refine the initial estimate.

- **Rationale for Intermediate COCOMO**:

- The basic model overlooks critical factors like reliability requirements, the use of modern development tools (CASE tools), and the volume of data to be processed.
- These factors significantly impact the actual effort and development time. For instance, using modern programming practices can reduce effort, while stringent reliability demands increase it.
- Intermediate COCOMO recognizes these influences, providing a more accurate estimation.
- **The Role of Cost Drivers**:
- After obtaining an initial estimate using Basic COCOMO, this estimate is refined by applying these 15 cost drivers.
- These drivers are categorized into four main types:
- **Personnel attributes**: Analyst capability (ACAP), virtual machine experience (VEXP), programmer capability (PCAP), programming language experience (LEXP), application experience (AEXP).
- **Product attributes**: Reliability requirement (RELY), database size (DATA), product complexity (CPLX).
- **Computer attributes**: Execution time constraints (TIME), storage constraints (STOR), virtual machine volatility (VIRT), computer turnaround time (TURN).
- **Project attributes**: Modern programming practices (MODP), software tools (TOOL), required development schedule (SCED).
- Each of these 15 attributes receives a rating on a six-point scale (e.g., very low, low, nominal, high, very high, extra high), which corresponds to a specific multiplier value.
- These multiplier values can be greater than 1 (scaling up the effort) or less than 1 (scaling down the effort). For example, high analyst capability might have a multiplier less than 1, reducing the estimated effort.
- **Calculating Effort Adjustment Factor (EAF)**:
- The `Effort Adjustment Factor (EAF)` is calculated by multiplying the values of all applicable cost drivers for a given project.
- The EAF typically ranges from 0.9 to 1.4, indicating the overall adjustment to the basic estimate.
- **Intermediate COCOMO Effort and Time Equations**:
- The refined effort is calculated using the formula:

```
Effort = EAF * c * (size)^k
```

where:
* `EAF` is the Effort Adjustment Factor.
* `c` is a constant based on the developmental mode (e.g., 3.2 for organic, 3.0 for semi-detached, 2.8 for embedded).
* `size` is the product size in thousands of Delivered Source Instructions (KDSI).
* `k` is a constant for the given mode (e.g., 1.05 for organic, 1.12 for semi-detached, 1.20 for embedded).
* The development time calculation uses this refined effort in the same way as Basic COCOMO:

```
Nominal Development time = 2.5 * (Effort)^exponent
```

where:
* `2.5` is a constant for all modes.
* `exponent` is based on the mode (e.g., 0.38 for organic, 0.35 for semi-detached, 0.32 for embedded).

- **Examples of Intermediate COCOMO Application**:
- **Analyst Capability**: If an initial estimate is 32.6 staff months and the analyst capability is rated "high" (multiplier 0.80), the adjusted effort becomes 32.6 * 0.80 = 26.8 staff months.
- **10 KLOC Organic Project**: For a 10 KLOC organic project, initial Basic COCOMO estimates might be 26.9 Person-Months and 8.7 Months development time. If specific attributes like analyst capability (low, 1.19), application experience (low, 1.13), programmer capability (low, 1.17), and programming experience (high, 0.95) are considered, the EAF is 1.19 * 1.13 * 1.17 * 0.95 = 1.49. The refined effort becomes 26.9 * 1.49 = 40 Person-Months, leading to a revised development time of 10.2 Months and approximately 4 people.
- **Flight Control System (Embedded Mode)**: For a mission-critical system with 319,000 DSI in embedded mode, requiring very high reliability (RELY=1.40), the effort calculation directly incorporates this multiplier:

```
Effort = 1.40 * 3.6 * (319)^1.20 = 5093 PM (approx.)
```

This refined effort then leads to a duration of 38.4 months and staffing of 133 people.

* **Embedded Software System with Multiple Attributes**: If Basic COCOMO predicts 45 person-months, and attributes like reliability (1.15), storage (1.21), time (1.10), and tools (1.10) are considered, the EAF is 1.15 * 1.21 * 1.10 * 1.10 = 1.69. The refined effort is 45 * 1.69 = 76 person-months. If the cost per person-month is Rs. 50,000, the total cost is 76 * 50,000 = Rs. 3,800,000.

2. Shortcomings of Basic and Intermediate COCOMO

Despite the improvements offered by Intermediate COCOMO, both Basic and Intermediate models have inherent limitations, particularly in the context of modern software development practices:

- **Calibration Sensitivity**: COCOMO models need to be calibrated to an organization's specific environment for better accuracy. They are highly sensitive to parameter changes; even minor adjustments can lead to significant differences in effort estimates (e.g., a person-year difference for a 10 KLOC project).
- **Broad Brush Model**: They are considered "broad brush" models, meaning they can generate significant errors if not applied carefully or if the project deviates substantially from typical assumptions.
- **Lack of Modern Practice Consideration**: These models were developed assuming a waterfall process and "from-scratch" development. They do not adequately account for modern practices such as:
- Software reuse
- Application generation programs
- Object-oriented approaches
- Application engineering (e.g., reverse engineering, application translation)
- The need for rapid development methodologies.
- **Homogeneous Entity Assumption**: A critical drawback is that both models treat a software product as a single, homogeneous entity. This means they assume all parts of a project share the same characteristics (e.g., development mode, reliability requirements).
- However, most large systems are composed of several smaller, heterogeneous sub-systems. Some sub-systems might be organic, others embedded, and some might require high reliability while others do not. The basic and intermediate models cannot differentiate these varying characteristics within a single project.

3. Complete COCOMO: Decomposing Complex Systems

To overcome the limitation of treating software as a single homogeneous entity, the Complete COCOMO (also known as Detailed COCOMO) model was introduced. This model provides a more granular and accurate approach for large, complex systems.

- **Addressing Heterogeneity**:
- Complete COCOMO recognizes that large systems are typically made up of diverse sub-systems, each potentially having different development modes, reliability requirements, and other attributes.
- Instead of applying a single set of parameters to the entire project, it allows for individual estimation of each component.
- **Estimation Process**:
- The core principle is to estimate the cost of each sub-system separately. This estimation can be done using either the Basic COCOMO or Intermediate COCOMO model, tailored to the specific characteristics of that sub-system.
- Once individual sub-system costs are estimated, they are summed up to obtain the total cost for the entire project.
- This approach significantly reduces the margin of error in the final estimate because it accounts for the unique nature of each component.
- **Example: Management Information System (MIS)**:
- Consider developing an MIS for an organization with offices across the country. Such a system is inherently complex and heterogeneous.
- It might consist of:
- A **database part**: which could be treated as a semi-detached system due to its structured nature and moderate complexity.
- A **Graphical User Interface (GUI) part**: often considered an organic component, as it might involve more straightforward, familiar development.
- A **communication part**: essential for connecting distributed offices, which would likely be treated as an embedded system due to its stringent real-time and reliability requirements.
- Under Complete COCOMO, the cost for each of these three distinct components (database, GUI, communication) would be estimated separately, using appropriate COCOMO parameters for their respective types.
- Finally, the estimated costs of these individual components would be added together to derive the overall cost for the entire MIS.

Key Takeaways

- [ ] Basic COCOMO estimates effort and time based primarily on product size.
- [ ] Intermediate COCOMO refines basic estimates by incorporating 15 cost drivers (effort multipliers).
- [ ] Cost drivers are categorized into personnel, product, computer, and project attributes.
- [ ] The Effort Adjustment Factor (EAF) is the product of all applicable cost driver values.
- [ ] Intermediate COCOMO uses EAF to adjust the base effort calculation: `Effort = EAF * c * (size)^k`.
- [ ] Development time is calculated using the refined effort from Intermediate COCOMO.
- [ ] Basic and Intermediate COCOMO models struggle with modern development practices and assume homogeneous projects.
- [ ] Complete COCOMO addresses heterogeneity by estimating each sub-system separately and summing their costs.
- [ ] The choice of COCOMO model depends on the project's complexity and the desired level of estimation accuracy.

Conclusion

The evolution from Basic to Intermediate and then to Complete COCOMO reflects a growing understanding of the multifaceted nature of software project estimation. While Basic COCOMO offers a quick, initial approximation, Intermediate COCOMO provides a more realistic picture by factoring in critical project-specific attributes through its system of cost drivers. This allows project managers to account for the unique challenges and advantages of their teams, technologies, and product requirements. However, for truly large and complex systems, where different components may exhibit vastly different characteristics, Complete COCOMO stands out as the most robust approach. By enabling the decomposition of a project into its heterogeneous sub-systems and estimating each individually, Complete COCOMO significantly enhances the accuracy of overall project cost and effort predictions. Mastering these models is essential for any project manager aiming to deliver projects on time and within budget in today's dynamic software landscape.